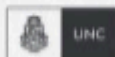


CERTIFICACIÓN UNIVERSITARIA



Manual para el alumno

Gitlab CI / Github
Actions I



**WE WANT
SKILLS!**

mundosE

Manual para el Alumno: Gitlab CI / Github Actions I.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Automatización de pipelines.....	3
2. ¿Qué es un pipeline?.....	4
3. GitHub Actions.....	5
3.1 Coordinator.....	5
3.2 Runners.....	5
3.3 Executors.....	5
3.4 Seguridad.....	6
4. Runners y control de ejecución.....	6
5. Jobs, variables y caché.....	7
5.2 Tipos de variables.....	7
5.3 Estrategia de caché.....	8
5.4 Mejores prácticas.....	8
Bibliografía.....	9

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En él encontraremos un resumen de cada tema tratado en el encuentro, así como también recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Gitlab CI / Github Actions I

Objetivo del encuentro:

Al finalizar este encuentro comprenderás la relevancia de la Integración Continua (CI) dentro de DevOps, reconocerás la anatomía de un pipeline en **GitLab CI** y serás capaz de crear, ejecutar y depurar un archivo **.gitlab-ci.yml** con las etapas esenciales (*build* y *test*) en tu propio proyecto, interpretando con criterio los resultados que muestran los jobs y el DAG en la pestaña **CI/CD → Pipelines**.

Contenido a desarrollar:

- Automatización de pipelines
- ¿Qué es un pipeline?
- GitHub Actions
- Runners y control de ejecución
- Jobs, variables y caché

1. Automatización de pipelines

La automatización de pipelines es el alma de la filosofía DevOps porque traslada las tareas repetitivas—compilación, pruebas, análisis de seguridad, empaquetado y despliegue—del plano manual al plano orquestado.

Con **GitLab CI** esto se logra codificando el proceso en un único archivo YAML (`.gitlab-ci.yml`) que vive en la raíz del repositorio. Al aplicar *pipeline as code* el flujo evoluciona con el código, garantiza auditoría y habilita **rollback** seguro.

Cada *push* o *merge request* dispara una línea de “montaje virtual” que proporciona *feedback* en cuestión de minutos y elimina el **integration hell**.

En términos de negocio, la automatización se refleja en métricas: según el *State of DevOps Report*, los equipos que practican CI/CD despliegan 46 × más rápido y sufren 5 × menos fallos de producción.

Al fallar un pipeline, el estado rojo es visible para todo el equipo y se desencadena aprendizaje colectivo.

Culturalmente, el pipeline refuerza los principios Lean: *Flujo*, *Feedback* y *Mejora Continua*.

No podemos dejar de pensar en costos ocultos de procesos manuales (compilaciones locales fallidas, esperas entre equipos, dependencia de “gurús”) y traduciremos esas ineficiencias a horas-hombre salvadas cuando se adopta CI.

Recursos para profundizar:

- Artículo: *Ultimate guide to CI/CD – Fundamentals to advanced implementation*
<https://about.gitlab.com/blog/2025/01/06/ultimate-guide-to-ci-cd-fundamentals-to-advanced-implementation/>
- Informe PDF: *Accelerate State of DevOps Report 2024*
https://services.google.com/fh/files/misc/2024_final_dora_report.pdf
- Libro gratuito: *Mastering GitLab 12* (Packt, descarga libre)
<https://www.packtpub.com/free-ebook/mastering-gitlab-12/9781789531282>

2. ¿Qué es un pipeline?

En GitLab un pipeline es un **DAG** (grafo dirigido acíclico) compuesto por **stages** (etapas secuenciales) y **jobs** (tareas paralelizables).

El pipeline se define íntegramente en `.gitlab-ci.yml`. Cuando ocurre un evento (commit, tag, merge request, scheduler, trigger manual) GitLab lee ese YAML y genera una representación gráfica.

Cada stage agrupa jobs afines—por ejemplo **build**, **test**, **security**, **deploy**.

GitLab ejecuta todos los jobs de un stage en paralelo (si hay runners suficientes) y no avanza al siguiente hasta que todos terminan en verde.

Flujo habitual:

1. **build** – compilación reproducible, empaquetado; artefactos como binarios o imágenes Docker.
2. **test** – unit e integración; se generan reportes JUnit y cobertura/LCOV.
3. **security** – análisis SAST/DAST, Container Scanning, Dependency Scanning.
4. **deploy** – promoción a *staging* o *production*, protegida con *manual gate*.

Algunas buenas prácticas a tener en cuenta: pipelines ≤ 10 min; pocos stages (3-5) para claridad; artefactos ligeros; uso de plantillas `include:` para evitar duplicación de YAML.

Recursos para profundizar:

- Documentación: *GitLab CI/CD YAML Reference*
<https://docs.gitlab.com/ee/ci/yaml/>
- Guía Docs: *Optimizing pipelines with needs:*
<https://docs.gitlab.com/ee/ci/yaml/#needs>
- Ejemplos: *Official .gitlab-ci.yml templates*
<https://gitlab.com/gitlab-org/gitlab/-/tree/master/lib/gitlab/ci/templates>

3. GitHub Actions

GitLab CI separa **orquestación** y **ejecución**.

La orquestación la asume el **Coordinator** interno de GitLab: detecta `.gitlab-ci.yml`, construye el pipeline y monitoriza su estado.

La ejecución recae en **Runners**, agentes externos que pueden ser *shared* (compartidos por varios proyectos) o *specific* (ligados a un proyecto/grupo).

3.1 Coordinator

Tras un *push*, el Coordinator:

1. Lee `.gitlab-ci.yml`.
2. Expande variables y plantillas `include:`.
3. Valida sintaxis; si falla, marca el pipeline como `failed` en fase `config`.
4. Genera un plan de etapas y jobs (DAG).
5. Inserta cada job en la tabla `ci_builds`, indicando `pending`.

3.2 Runners

Un Runner (binario `gitlab-runner`) se registra con `gitlab-runner register`, recibiendo un token. Cada *n segundos* llama a `POST /api/v4/jobs/request` con sus `tags`.

Si hay un job compatible, recibe:

- meta del job (ID, comando, variables de entorno `CI_*`),
- URL del repositorio y commit SHA,
- JWT corto `CI_JOB_TOKEN` (scope limitado).

El Runner clona el repo (profundidad 50 por defecto), crea un *build directory*, ejecuta el **executor** seleccionado (shell, docker, kubernetes, docker-machine) y escribe logs con `PATCH /jobs/:id/trace`. Finalizado, envía artefactos (`POST /jobs/:id/artifacts`) y estado (`PUT /jobs/:id`).

3.3 Executors

- **shell** – usa `/bin/bash`; rápido, sin aislamiento.
- **docker** – crea contenedor; imagen declarada con `image:`.
- **kubernetes** – lanza Pod por job; escala masiva.
- **docker-machine** – crea VM on-demand; autoscaling en cloud.

3.4 Seguridad

Variables protegidas (**masked**, **protected**) solo se exponen en pipelines sobre ramas protegidas. El token **CI_JOB_TOKEN** expira al terminar el job y puede autenticarse contra API y Package Registry.

Recursos para profundizar:

- Docs: How GitLab Runner works
<https://docs.gitlab.com/runner/#how-gitlab-runner-works>
- Tutorial: Securing CI with masked variables
<https://docs.gitlab.com/ee/ci/variables/#masked-variables>

4. Runners y control de ejecución

GitLab.com otorga 400 min gratuitos de runners compartidos por mes en su SaaS; en self-managed no hay límites.

Para requisitos especiales—hardware GPU, redes privadas—registras **specific runners**. El registro interactivo solicita URL, token y etiquetas (**tags:**). Verás cómo los **tags** dirigen jobs de Windows o Android a runners adecuados.

El archivo **config.toml**: sección **[runners.docker]** sirve para definir la imagen base, **volumes**, **pull_policy**, caché con MinIO; y **[runners.cache]** para almacenar dependencias en S3/GCS y acelerar builds.

El parámetro **parallel** que permite a un runner ejecutar varios jobs simultáneamente y la propiedad **limit** que impone cupo.

Con **autoscaling** vía **docker-machine** o **Kubernetes executor** crearás instancias “just-in-time” reduciendo costos.

Es posible la retención de artefactos que expiren con (**expire_in:**) y las reglas de caché (**cache:key:**) para invalidar al cambiar **package-lock.json**.

Recursos para profundizar:

- Guía: Registering runners <https://docs.gitlab.com/runner/register/>
- Repo: gitlab-org/charts/gitlab-runner (Helm chart Kubernetes)
<https://gitlab.com/gitlab-org/charts/gitlab-runner>

5. Jobs, variables y caché

Un **job** es la unidad mínima de ejecución dentro de GitLab CI. Cada job debe pertenecer a un **stage** y contener al menos una directiva **script:** con los comandos a ejecutar. Otras claves clave son **image:** (para fijar un contenedor base), **tags:** (para filtrar runners), **artifacts:** (para preservar archivos) y **cache:** (para acelerar builds futuros).

5.1 Anatomía de un job

```
compile_backend:           # nombre único y descriptivo
  stage: build              # etapa a la que pertenece
  image: maven:3.9-jdk-17   # contenedor base (Docker executor)
  before_script:            # comandos comunes previos (opcional)
    - echo "Compilación iniciada en $(date)"
  script:                   # comandos principales (obligatorio)
    - mvn -B clean package --fail-at-end
  after_script:             # limpieza o notificación (opcional)
    - echo "Fin del job"
  artifacts:                # archivos que se conservarán
    paths:
      - target/*.jar
    expire_in: 1 week
  cache:                    # contenido a cachear
    key: "$CI_COMMIT_REF_SLUG-maven"
    paths:
      - ~/.m2/repository
```

- **before_script / after_script:** se ejecutan en cada job (o de forma global en la raíz del YAML). Útiles para preparar variables, validar versiones o enviar métricas.
- **image:** permite un entorno coherente. Evita la “works on my machine”.
- **artifacts:** guarda binarios, reportes JUnit (**reports:junit:**), cobertura (**reports:coverage_report:**) o resultados de análisis. Se pueden descargar desde la UI y consumir en stages posteriores añadiendo **dependencies:** o **needs:**.

5.2 Tipos de variables

GitLab crea variables predefinidas (**CI_COMMIT_SHA**, **CI_PROJECT_PATH_SLUG**, **CI_PIPELINE_SOURCE**) y permite definir variables personalizadas tanto en el YAML como en la interfaz:

- **Variables protegidas:** sólo se exponen en ramas protegidas. Ideal para claves de despliegue.

- **Variables enmascaradas (masked):** su valor se oculta en logs. GitLab exige regex de validación.
- **Variables de grupo:** compartidas por varios proyectos, útiles para tokens corporativos.

Ejemplo de uso:

```
variables:
  DOCKER_REGISTRY: registry.gitlab.com
  DOCKER_IMAGE: "$CI_PROJECT_PATH:$CI_COMMIT_SHORT_SHA"
deploy:
  stage: deploy
  script:
    - docker login -u gitlab-ci-token -p $CI_JOB_TOKEN $DOCKER_REGISTRY
    - docker push $DOCKER_IMAGE
```

En este caso, el token temporal **CI_JOB_TOKEN** hereda permisos restringidos y evita exponer credenciales permanentes.

5.3 Estrategia de caché

El caché acelera la instalación de dependencias y compila solo los cambios.

- **key::** identifica la versión del caché. Puede incluir variables o el hash de archivos (**\$CI_COMMIT_REF_SLUG-\$CI_PROJECT_NAME**).
- **paths::** rutas a guardar.
- **policy::** **pull-push** (default), **pull**, **push**.

GitLab genera un archivo tar y lo sube al backend de caché (S3, GCS o filesystem). Para invalidar el caché al cambiar dependencias, agrega hash de archivo:

```
cache:
  key: "${CI_COMMIT_REF_SLUG}-${sha256sum package-lock.json | cut -d' ' -f1}"
  paths:
    - node_modules/
```

5.4 Mejores prácticas

1. **Fail-fast:** activa **set -e** o **errexit** en scripts para abortar en el primer error.
2. **Artefactos vs Caché:** artefactos viajan entre stages, caché acelera futuras pipelines; no los confundas.
3. **Jobs paralelos:** usa **parallel:** o divide tests en jobs separados para reducir tiempo total.
4. **Resource_group:** evita “**race conditions**” en despliegues

(`resource_group: production`).

5. **Plantillas:** extrae jobs comunes a `.gitlab-ci.yml` de plantilla y `include: template:...` para DRY.

Recursos para profundizar:

- Docs: *Caching dependencies* <https://docs.gitlab.com/ee/ci/caching/>
- Guía Docs: *Masked and protected CI/CD variables*
<https://docs.gitlab.com/ee/ci/variables/#masked-and-protected-variables>

Bibliografía

- **GitLab Docs.** "CI/CD pipelines – Conceptos y Ejemplos." <https://docs.gitlab.com/ci/pipelines/> GitLab Docs
- **GitLab Docs.** "Get Started with GitLab CI/CD." <https://docs.gitlab.com/ci/> GitLab Docs